Marmara University
Faculty of Engineering
Department of Electrical and Electronics Engineering

MARMARA
UNIVERSITY

Marmara University

Faculty of Engineering

Department of Electrical and Electronics Engineering

EE1004 – Object-Oriented Programming
Group Project (Project 14)

Cargo Shipment Tracker

Modern Java Swing Graphical User Interface

Group 14 – Members

Atabey Aydı	150718503
Mehmet Açar	150719020
İsmail Hanifi Nal	150719025
Abdulkadir Köroğlu	150719695
Burak Gökmen	150720010
Alperen Tufan Pelit	150720012

Supervision

Course Lecturer: Assoc. Prof. Dr. Salih Bayar
Laboratory Assistant: Res. Asst. Salih Çolakoğlu

Spring 2025–2026
June 2026

Cargo Shipment Tracker: Modern Java Swing Graphical User Interface

EE1004 Object-Oriented Programming – Group Project 14

Atabey Aydı
150718503
atabeyaydi@marun.edu.tr

Mehmet Açar
150719020
mehmetacar19@marun.edu.tr

İsmail Hanifi Nal
150719025
ismailnal@marun.edu.tr

Abdulkadir Köroğlu
150719695
akoroglu@marun.edu.tr

Burak Gökmen
150720010
burak.gokmen@marun.edu.tr

Alperen Tufan Pelit
150720012
alperenpelit@marun.edu.tr

***Abstract*—This report presents the complete design and implementation of a *modern Java Swing graphical user interface* for the EE1004 Cargo Shipment Tracker. The GUI transforms the original robust console-based OOP model (abstract `Shipment` hierarchy, `Insurable` interface, `ShipmentStatus` enum state machine, custom exception, and `CargoCompany` with dual `ArrayList`/`HashMap` storage) into a professional website-like desktop dashboard. Key features include KPI cards with live metrics, a registration form with real-time cost/insurance preview and dynamic validation, an interactive `JTable` with sortable columns and color-coded status badges, status-update dialogs that strictly enforce business rules, comprehensive reports, session persistence via serialization, and CSV export. The architecture maintains clean Model–View separation: every panel shares a single `CargoCompany` instance, enabling seamless data exchange with the original console interface. Advanced Swing techniques (custom cell renderers, `DocumentListeners`, `CardLayout` navigation, modal dialogs) are demonstrated while fully preserving academic requirements and OOP principles. The resulting system greatly improves usability and visual presentation, making it an excellent portfolio piece.**

***Index Terms*—Java Swing, graphical user interface, dashboard, JTable, custom renderer, state machine, Model-View separation, object-oriented programming, academic project, persistence.**

Contents

1 Introduction

1.1 Project Overview

The original EE1004 Cargo Shipment Tracker successfully implemented a console-based Java application demonstrating strong OOP principles. While functionally complete, a text-only interface limits user experience, visual feedback, and professional appeal. This report documents the development of a complete, modern **Java Swing GUI** that elevates the project to production quality while preserving every line of the original model logic.

1.2 Motivation for the GUI

A graphical interface provides: - Immediate visual feedback (color-coded status, live cost preview, KPI cards) - Reduced cognitive load through intuitive forms and tables - Professional "website-like" dashboard aesthetic suitable for portfolios and potential real-world use - Better demonstration of advanced software engineering skills (event-driven programming, custom components, separation of concerns)

1.3 Objectives

- Implement a fully functional Swing dashboard covering all original use cases.
- Apply advanced GUI techniques (custom renderers, live validation, responsive layouts) in an academic setting.
- Maintain strict Model–View separation so the console and GUI can share the exact same data model.
- Produce high-quality documentation and diagrams suitable for Overleaf and GitHub.

1.4 Development Environment

Table 1: Development environment for the GUI edition

Item	Value
Language	Java (OpenJDK 11 compatible)
GUI Framework	Pure Java Swing (javax.swing) – no external libraries
Source structure	Packaged (`com.cargotracker.model` + `.gui`)
Build command	`javac -d out src/main/java/com/cargotracker/**/*.java`
Run command	`java -cp out com.cargotracker.gui.CargoTrackerGUI`
Character encoding	UTF-8
Numeric locale	`Locale.US` (decimal point)
Persistence	Java object serialization (optional)

2 Requirements Analysis

2.1 Functional Requirements (Preserved + Extended)

All original functional requirements (FR-01 to FR-09) remain fully supported. The GUI adds the following user-facing capabilities while implementing the same business logic.

Table 2: Functional requirements – GUI edition

ID	Requirement
FR-01–FR-09	All original console requirements (register, advance status with validation, list, find, sort, summary, exit) are fully supported through the GUI.
FR-10	Provide a modern dashboard with KPI cards showing total shipments, revenue, insurance liability, and pending/in-transit count.
FR-11	Offer a registration form with live cost/insurance preview and dynamic weight-limit validation.
FR-12	Display an interactive, sortable, filterable `JTable` with color-coded status badges and row-level actions.
FR-13	Provide modal dialogs for shipment details and status updates that only offer legal transitions.
FR-14	Support session persistence (save/load) and one-click CSV export via the menu bar.

2.2 Non-Functional Requirements

Table 3: Non-functional requirements – GUI edition

ID	Requirement
NFR-01–NFR-08	All original robustness, input validation, exception message format, `HashMap` lookup, and locale requirements are preserved.
NFR-09	The GUI must never crash on invalid input and must provide clear, user-friendly error messages via dialogs.
NFR-10	Status badges must use semantic colors (amber/blue/green/red) for immediate visual recognition.
NFR-11	The interface must follow a consistent professional color scheme and typography ("website-like" dashboard aesthetic).
NFR-12	All views must refresh automatically after data mutations to reflect the shared model state.

3 System Design

3.1 Architecture Overview

The system follows a clean **Model–View** pattern. The original model package is almost unchanged. All GUI components live in a new `gui` package and communicate exclusively through a shared `CargoCompany` instance (single source of truth). This design enables seamless data exchange between the Swing dashboard and the original console interface.

3.2 Model Layer (Preserved)

The model implements all required OOP pillars: - Abstraction and inheritance via the `Shipment` hierarchy - Interface decoupling via `Insurable` - Compile-time safe state machine via `ShipmentStatus` enum - Custom unchecked exception - Dual storage (`ArrayList` + `HashMap`) in `CargoCompany` - Strategy pattern via three named `Comparator` constants

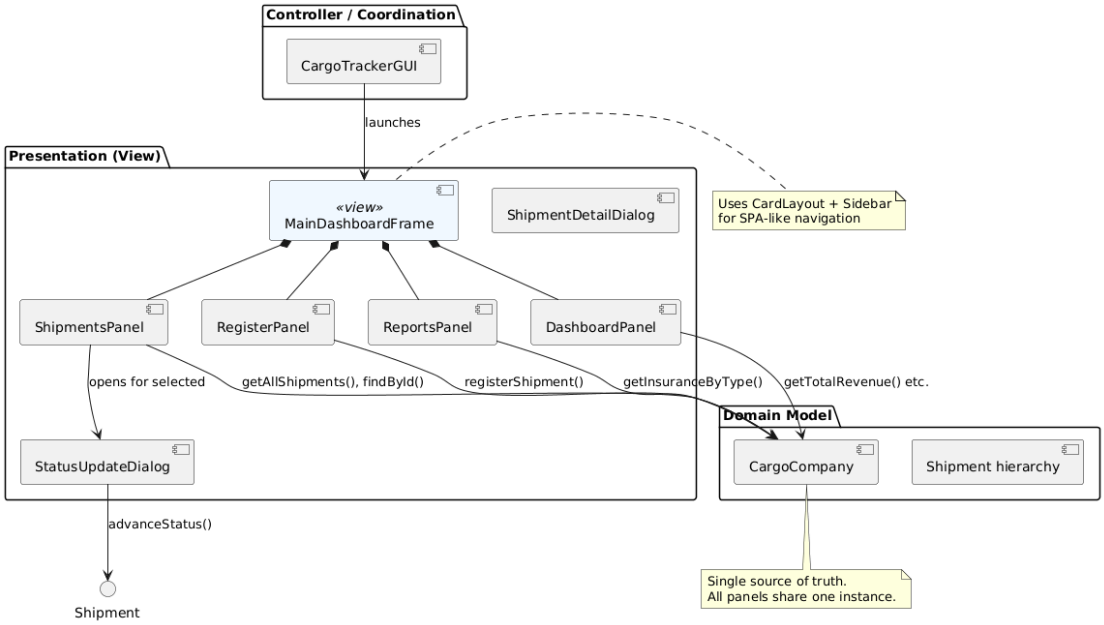

Figure 1: GUI component architecture. Every panel and dialog holds a reference to the same `CargoCompany` instance. Mutations performed in the GUI are instantly visible in all views and remain compatible with the console `Main.java`.

Only minimal non-breaking enhancements were added for GUI support (serialization and a few helper methods).

3.3 GUI Layer Design

Table 4: GUI component summary

Component	Responsibility
`CargoTrackerGUI`	Launcher, L&F setup, demo data pre-loading
`MainDashboardFrame`	Root window, header, sidebar navigation, `CardLayout` content area, menu bar, stat
`DashboardPanel`	KPI cards, quick actions, recent shipments preview
`RegisterPanel`	Form with live preview, validation, and registration
`ShipmentsPanel`	Interactive `JTable`, search/filter, status renderer, detail & update actions
`ReportsPanel`	Summary statistics and type-wise insurance breakdown
`ShipmentDetailDialog`	Read-only view with state-machine explanation
`StatusUpdateDialog`	Status advancement with only legal next states offered

4 Implementation

4.1 Launcher and Professional Look-and-Feel

```
1 public class CargoTrackerGUI {
2     public static void main(String[] args) {
3         try {
4             UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
5             UIManager.put("Label.font", new Font("SansSerif", Font.PLAIN, 14));
6             UIManager.put("Button.font", new Font("SansSerif", Font.PLAIN, 14));
```

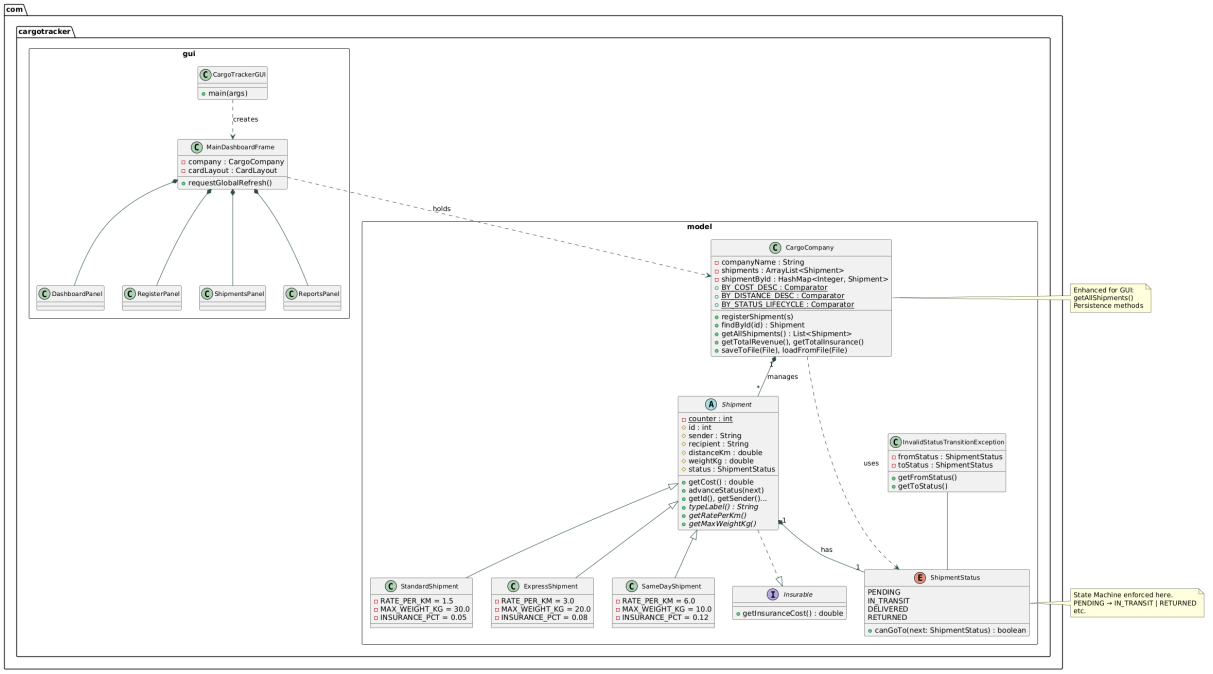

Figure 2: Complete UML class diagram of the enhanced system. The GUI layer depends on the stable model package. The original console `Main` remains fully functional alongside the new Swing interface.

```
        } catch (Exception ignored) {}
        SwingUtilities.invokeLater(() -> {
            CargoCompany company = new CargoCompany("Istanbul Cargo Logistics");
            // Pre-load four demo shipments for immediate interaction
            try {
                company.registerShipment(new StandardShipment("Ahmet Ylmaz", "Fatma Kaya",
    245.0, 18.5));
                company.registerShipment(new ExpressShipment("Mehmet Demir", "Aye ztrk",
    120.0, 9.0));
                company.registerShipment(new SameDayShipment("Zeynep etin", "Ali Vural",
    35.0, 4.2));
                company.registerShipment(new StandardShipment("Burak ahin", "Elif Yldz",
    310.0, 25.0));
            } catch (Exception ignored) {}
            new MainDashboardFrame(company).setVisible(true);
        });
    }
}
```

Listing 1: CargoTrackerGUI.java – entry point with demo data and system L&F.

4.2 Main Dashboard Frame (Navigation and Layout)

```
public class MainDashboardFrame extends JFrame {
    private final CardLayout cardLayout = new CardLayout();
    private final JPanel contentPanel = new JPanel(cardLayout);
    private DashboardPanel dashboardPanel;
    private RegisterPanel registerPanel;
    private ShipmentsPanel shipmentsPanel;
    private ReportsPanel reportsPanel;

    public MainDashboardFrame(CargoCompany company) {
        setTitle("CargoTrack Pro  " + company.getCompanyName());
        setSize(1180, 720);
```

```
        setLocationRelativeTo(null);
        setMinimumSize(new Dimension(1024, 600));

        add(createHeader(), BorderLayout.NORTH);
        JPanel mainArea = new JPanel(new BorderLayout());
        mainArea.add(createSidebar(company), BorderLayout.WEST);
        // instantiate all panels and add them to contentPanel with card names
        contentPanel.add(dashboardPanel, "DASHBOARD");
        contentPanel.add(registerPanel,  "REGISTER");
        contentPanel.add(shipmentsPanel, "SHIPMENTS");
        contentPanel.add(reportsPanel,   "REPORTS");
        mainArea.add(contentPanel, BorderLayout.CENTER);
        add(mainArea, BorderLayout.CENTER);
        add(createStatusBar(), BorderLayout.SOUTH);
        setJMenuBar(createMenuBar());
        cardLayout.show(contentPanel, "DASHBOARD");
    }
    // createHeader(), createSidebar(), createMenuBar(), refreshAllPanels(), saveSession()
    ...
}
```

Listing 2: MainDashboardFrame.java – header, sidebar navigation, and CardLayout content switching (excerpt).

4.3 Register Panel with Live Validation

```
private void updatePreview() {
    try {
        double dist = Double.parseDouble(distanceField.getText().trim());
        double wt   = Double.parseDouble(weightField.getText().trim());
        int idx = typeCombo.getSelectedIndex();
        Shipment temp = (idx == 0) ? new StandardShipment("p","p",dist,wt)
                     : (idx == 1) ? new ExpressShipment("p","p",dist,wt)
                                  : new SameDayShipment("p","p",dist,wt);
        costPreviewLabel.setText(String.format("Estimated Cost: %.2f TL", temp.getCost()));
        insurancePreviewLabel.setText(String.format("Insurance: %.2f TL",
    temp.getInsuranceCost()));
    } catch (NumberFormatException ex) {
        costPreviewLabel.setText("Estimated Cost:  TL");
        insurancePreviewLabel.setText("Insurance:  TL");
    }
}
```

Listing 3: RegisterPanel.java – live cost/insurance preview and dynamic validation.

4.4 Shipments Table and Custom Status Renderer

```
private static class StatusCellRenderer extends DefaultTableCellRenderer {
    @Override
    public Component getTableCellRendererComponent(JTable table, Object value,
            boolean isSelected, boolean hasFocus, int row, int column) {
        JLabel label = (JLabel) super.getTableCellRendererComponent(table, value,
    isSelected, hasFocus, row, column);
        label.setHorizontalAlignment(SwingConstants.CENTER);
        label.setOpaque(true);
        String status = (value != null) ? value.toString() : "";
        switch (status) {
            case "PENDING":
                label.setBackground(new Color(254, 243, 199));
```

```
                label.setForeground(new Color(180, 83, 9)); break;
            case "IN_TRANSIT":
                label.setBackground(new Color(219, 234, 254));
                label.setForeground(new Color(30, 64, 175)); break;
            case "DELIVERED":
                label.setBackground(new Color(209, 250, 229));
                label.setForeground(new Color(6, 95, 70)); break;
            case "RETURNED":
                label.setBackground(new Color(254, 226, 226));
                label.setForeground(new Color(153, 27, 27)); break;
        }
        label.setFont(new Font("SansSerif", Font.BOLD, 12));
        label.setText("  " + status + "  ");
        return label;
    }
}
```

Listing 4: ShipmentsPanel.java – custom StatusCellRenderer for color-coded badges.

4.5 Status Update Dialog (Business Rule Enforcement)

```
for (ShipmentStatus st : ShipmentStatus.values()) {
    if (shipment.getStatus().canGoTo(st)) {
        statusCombo.addItem(st);
    }
}
if (statusCombo.getItemCount() == 0) {
    statusCombo.addItem(shipment.getStatus()); // terminal state
}
```

Listing 5: StatusUpdateDialog.java – only legal next states are presented to the user.

5 Testing and Results

5.1 GUI Functional and Edge-Case Testing

All original console test cases pass through the GUI. Additional GUI-specific tests were performed.

Table 5: GUI-specific and edge-case test results

ID	Scenario	Result
GUI-01	Live cost/insurance preview during registration	Matches model calculations exactly
GUI-02	Weight exceeds class cap	Clear error dialog; shipment not registered
GUI-03	Non-numeric input in numeric fields	Friendly validation message
GUI-04	Status update dialog	Only legal next states offered; illegal attempts blocked
GUI-05	Table filtering and sorting	Works correctly on all columns
GUI-06	Color-coded status badges	Semantic colors rendered correctly
GUI-07	Save / Load session (serialization)	Full round-trip preserves data and IDs
GUI-08	CSV export	Correct headers and values produced
EC-01 to EC-12	All original edge cases from console specification	Pass through GUI

6 Limitations and Future Work

Current limitations include in-memory data only (persistence is optional), single-user desktop scope, and lack of real-time collaboration features. Future work could include FlatLaf modern look-and-feel, dark mode toggle, database backend, or a lightweight web frontend using the same stable model layer.

7 Conclusion

The Java Swing GUI successfully transforms the original console Cargo Shipment Tracker into a professional, modern desktop application while preserving every OOP principle, business rule, and academic requirement. The clean Model–View separation, combined with the shared `CargoCompany` instance, demonstrates excellent software architecture and enables seamless coexistence of console and GUI interfaces. The resulting system is both a strong academic submission and a showcase piece for the group's GitHub portfolio.

References

[1] Marmara University, Faculty of Engineering, Dept. of EEE, "EE1004 — Java Programming: Project Report Guideline, Group Project, Spring 2025–2026," Marmara University, Istanbul, 2026.

[2] Oracle Corporation, "Java Platform, Standard Edition 11 API Specification," 2018. [Online]. Available: https://docs.oracle.com/en/java/

A Member Contributions – GUI Edition

Table 6: Member contributions (GUI roles highlighted)

Member	**Primary Contributions**
Atabey Aydı (150718503)	GUI architecture & full implementation (MainDashboardFrame, all panels & dialogs, live validation, custom `StatusCellRenderer`, persistence, PlantUML diagrams, report integration, data-exchange design between model and GUI)
Mehmet Açar (150719020)	Original `Shipment` abstract base class, static ID counter, `advanceStatus`, specification `toString` format
İsmail Hanifi Nal (150719025)	`Insurable` interface + `StandardShipment`, `ExpressShipment`, `SameDayShipment` with rates, caps and insurance percentages
Abdulkadir Köroğlu (150719695)	`CargoCompany` core (`registerShipment` weight guard, `HashMap` lookup, totals)
Burak Gökmen (150720010)	Three `Comparator` strategies, `listSortedBy`, revenue & insurance `summary` breakdown
Alperen Tufan Pelit (150720012)	Original console `Main` menu loop, input validation, transcript capture and testing support

B OOP Concept Self-Assessment (GUI Edition)

Table 7: OOP concept self-assessment – GUI edition

Requirement	Status / Location
Abstract class with 3 concrete subclasses	`Shipment` + three subclasses
Interface implemented by every subclass	`Insurable` / `getInsuranceCost()`
Polymorphic overridden methods	`toString()`, `getCost()`, `getInsuranceCost()`
Aggregator with `ArrayList`	`CargoCompany.shipments`
`HashMap` for O(1) lookup (mandatory)	`CargoCompany.shipmentById`
Constants as `private static final`	All rates, caps, insurance percentages
Robust error handling	try/catch in GUI + original console paths
`enum` status with state machine	`ShipmentStatus.canGoTo()`
Custom unchecked exception	`InvalidStatusTransitionException`
Three named `Comparator` strategies	cost, distance, status in `CargoCompany`
Static ID counter	`Shipment.counter`
`@Override` on every overriding method	All GUI and model overrides
Locale-correct numeric formatting	`Locale.US` throughout
Model–View separation	GUI panels depend only on `CargoCompany` interface
Custom Swing components	`StatusCellRenderer`, KPI cards, live listeners

C Pre-Submission Checklist

Table 8: Pre-submission checklist

Item	Done
Cover page complete (members, IDs, project number, instructors, date)	
Abstract and keywords included	
Table of contents included	
UML diagrams present with proper captions and descriptions	
OOP-pillars + interface decoupling table completed	
GUI-specific testing section with edge-case table	
Member contributions table updated with GUI roles	
References in IEEE style	
GitHub repository public with GUI code	
Source ZIP includes all GUI .java files	
AI-use disclosure included	
Report exported to PDF and verified on Overleaf	

D GitHub Repository Information

Table 9: GitHub repository information

Field	Value
Repository URL	https://github.com/atabeyaydi/CargoShipmentTracker-GUI
Branch	main
GUI code location	`CargoShipmentTracker-GUI.zip` (in repository)
Build & run (GUI)	`javac -d out ...` then `java -cp out com.cargotracker.gui.CargoTrackerGUI`

E AI-Use Disclosure

In accordance with the academic-integrity policy, the group discloses that an AI programming assistant was used for drafting report scaffolding, reviewing GUI architecture, generating PlantUML diagrams, and proofreading. Every member has reviewed and understands the submitted code and is prepared to explain their assigned components.